



WSQ COURSEWARE

Agentic AI for Video Creation

Learner Guide

Course Code	TGS-2023036088
Technical Skills and Competencies	MED-MPN-4005-1.1 - Video Editing-4
Version	v3.0
Release Date	15 September 2026
Training Provider	Tertiary Infotech Academy Pte Ltd (UEN 201200696W)
Trainer	Dr Alfred Ang

Controlled document. Verify the version before use.

Document Control

Document	Version	Date	Owner	Change summary
Learner Guide	2.0	29 August 2026	Tertiary Infotech Academy Pte Ltd	Agentic-video workflow with n8n, generative media, approval and YouTube release controls
Learner Guide	3.0	15 September 2026	Tertiary Infotech Academy Pte Ltd	Replaced n8n with Hermes Agent, MiniMax M3, FRAME-CUT prompting, video skills, Kanban multi-agent orchestration and scheduled private YouTube publishing

Contents

Section	Page
1. Course Overview	3
2. Creative Strategy and End-to-End Video Production with Agentic AI	5
3. AI-Assisted Video Editing, Storytelling and Quality Assurance	19
4. Workflow Optimisation, Industry Compliance and Emerging Video Technologies	33
5. Hands-on Lab Guide	47
6. Hermes and YouTube Release Contract	63
7. Assessment Preparation	63
8. References	63

1. Course Overview

This Learner Guide supports an end-to-end Hermes Agent video-production workflow using MiniMax M3. Learners install and verify Hermes Desktop, engineer prompts with FRAME-CUT, route work to Remotion, Manim, Higgsfield-compatible requests and FFmpeg, create a reusable brand-video skill, coordinate research, production, review and upload agents through Kanban, and prepare a controlled scheduled release. The guide contains the detailed procedures intentionally omitted from the visual slide deck.

SAFETY BOUNDARY

All lab data are synthetic. YouTube visibility remains private during practice. Never place MiniMax keys, OAuth tokens or other credentials in prompts, lab files, screenshots or repositories.

1.1 Learning Outcomes

- LO1: Develop an editing strategy and work plan in Hermes Agent using MiniMax M3, structured video prompts and governed tool selection.
- LO2: Create and customise an evidence-backed video with Hermes skills, Remotion, Manim, Higgsfield-compatible requests and deterministic media checks.
- LO3: Orchestrate research, production, review and YouTube release agents through a durable Kanban board and controlled scheduled publishing.

1.2 Knowledge and Ability Map

Code	Competency statement	Primary evidence
K1	Hermes Desktop architecture, MiniMax M3 model routing and credential boundaries	Written response
K2	FRAME-CUT video prompt engineering and structured output contracts	Written response
K3	Video tool selection, Hermes skills and deterministic render evidence	Written response
K4	Multi-agent delegation, durable Kanban dependencies and review gates	Written response
K5	YouTube upload, disclosure, idempotency, cron and operational controls	Written response
A1	Install and verify Hermes Desktop, then connect MiniMax M3 without exposing credentials	Hermes execution and artifact evidence
A2	Prompt Hermes to plan and create a short video using a validated prompt contract	Hermes execution and artifact evidence
A3	Install or create video skills and route work to Remotion, Manim or Higgsfield	Hermes execution and artifact evidence
A4	Customise brand, tone and style through a reusable Hermes skill	Hermes execution and artifact evidence
A5	Build and operate a research-video-review-upload multi-agent Kanban workflow	Hermes execution and artifact evidence
A6	Prepare a private YouTube upload and schedule a controlled publish job	Hermes execution and artifact evidence

1.3 End-to-End Reference Architecture

1. Install Hermes Desktop from the official page, run diagnostics and connect MiniMax-M3 through a redacted provider contract.

2. Convert the creative brief into a structured FRAME-CUT prompt and validate the timed shot plan before generation.
3. Route each media requirement to a suitable tool or Hermes skill: Remotion, Manim, Higgsfield or FFmpeg.
4. Create a reusable brand-video skill that carries palette, typography, tone, motion and output checks.
5. Dispatch research, video and independent review roles with explicit context and immutable handoff evidence.
6. Use a durable Kanban dependency graph to gate the upload role behind review and a matching human approval hash.
7. Prepare a private YouTube upload and a paused Hermes cron job with idempotency and evidence-led enablement.

2. Hermes Agent Setup, MiniMax M3 and Video Prompt Engineering

LO1 - configure Hermes safely and convert creative intent into a machine-checkable video plan

TOPIC OPERATING PRINCIPLE

Every agent handoff uses named fields; every consequential transition has a deterministic gate; every release decision resolves to evidence.

2.1 Hermes Desktop runtime boundary

Mechanism

- 1. Download installer
- 2. Verify platform
- 3. Complete setup
- 4. Run diagnostics

platform	installer_source	hermes_version	doctor_status
Required state field for download installer	Required state field for verify platform	Required state field for complete setup	Required state field for run diagnostics

Evidence and control

MEASURE

setup pass = required diagnostics passed / required diagnostics

Failure mode: An unofficial installer or unsupported platform creates an untrusted runtime.

Control: Use the official desktop page and stop when hermes doctor reports a failed dependency.

Acceptance evidence: setup-evidence.json records official source, version and diagnostic status.

2.2 Shared desktop and CLI state

Mechanism

- 1. Open Desktop
- 2. Inspect profile
- 3. Open terminal
- 4. Compare configuration

profile	config_path	session_id	skill_root
Required state field for open desktop	Required state field for inspect profile	Required state field for open terminal	Required state field for compare configuration

Evidence and control

MEASURE state consistency = matching settings / checked settings
--

Failure mode: Desktop and CLI appear to use different models because separate profiles are active.

Control: Name the active profile and compare provider, model and skill directory before production.

Acceptance evidence: profile-check.json shows the same provider and model on both surfaces.

2.3 MiniMax M3 provider contract

Mechanism

- 1. Create account
- 2. Create API key
- 3. Configure provider
- 4. Select model

provider	base_url	model	credential_ref
Required state field for create account	Required state field for create api key	Required state field for configure provider	Required state field for select model

Evidence and control

MEASURE model handshake pass = successful tool-capable response

Failure mode: The learner selects an older OAuth-listed model while assuming it is M3.

Control: Use provider minimax with model MiniMax-M3 and verify the returned model identifier.

Acceptance evidence: model-check.json records MiniMax-M3 without recording the key.

2.4 Trial and quota verification

Mechanism

1. Open live offer
2. Read terms
3. Record expiry
4. Set budget

offer_seen_at	trial_end	quota	fallback_model
Required state field for open live offer	Required state field for read terms	Required state field for record expiry	Required state field for set budget

Evidence and control

MEASURE

budget headroom = available quota - reserved quota

Failure mode: A course promise about a one-day trial becomes stale or differs by account region.

Control: Treat the offer as time-sensitive; record live terms at sign-up and prepare a fallback.

Acceptance evidence: trial-checklist.md contains observed terms, date and learner decision.

2.5 Credential isolation

Mechanism

- 1. Create secret
- 2. Store securely
- 3. Reference variable
- 4. Redact evidence

secret_name	scope	provider	rotation_due
Required state field for create secret	Required state field for store securely	Required state field for reference variable	Required state field for redact evidence

Evidence and control

MEASURE secret exposure count = 0

Failure mode: A MiniMax or YouTube key is pasted into a prompt, screenshot or repository.

Control: Keep secrets in Hermes credential/config storage and publish placeholders only.

Acceptance evidence: secret scan reports zero live tokens in the lab package.

2.6 Model capability routing

Mechanism

- 1. Classify task
- 2. Check modality
- 3. Choose M3
- 4. Choose media tool

task_type	input_modality	model_id	tool_id
Required state field for classify task	Required state field for check modality	Required state field for choose m3	Required state field for choose media tool

Evidence and control

MEASURE routing yield = accepted outputs / routed calls

Failure mode: A language model is asked to render video bytes instead of planning and operating a media tool.

Control: Use M3 for reasoning and tool orchestration; use a video renderer or generator for media.

Acceptance evidence: tool-routing.csv links every task to a capable model or tool.

2.7 FRAME-CUT prompt contract

Mechanism

- 1. Define FRAME
- 2. Add CUT controls
- 3. Require schema
- 4. Validate output

format_role_assets	motion_environment	continuity_unwanted	technical_output
Required state field for define frame	Required state field for add cut controls	Required state field for require schema	Required state field for validate output

Evidence and control

MEASURE prompt validity = required fields present / required fields

Failure mode: A cinematic prompt omits duration, aspect ratio, continuity or acceptance criteria.

Control: Use FRAME-CUT and validate the returned shot plan before generation.

Acceptance evidence: prompt-contract.json passes schema and duration checks.

2.8 Format and finish

Mechanism

- 1. Choose channel
- 2. Set duration
- 3. Set aspect
- 4. Set delivery codec

channel	duration_s	aspect_ratio	codec
Required state field for choose channel	Required state field for set duration	Required state field for set aspect	Required state field for set delivery codec

Evidence and control

MEASURE

delivery compliance = passed technical fields / required fields

Failure mode: The composition is correct but cannot be uploaded in the intended channel format.

Control: Freeze delivery properties before shot generation and assembly.

Acceptance evidence: video-spec.json contains approved format and finish values.

2.9 Role, references and assets

Mechanism

- 1. Assign role
- 2. Attach brief
- 3. Register sources
- 4. List assets

role	brief_uri	source_ids	asset_ids
Required state field for assign role	Required state field for attach brief	Required state field for register sources	Required state field for list assets

Evidence and control

MEASURE asset coverage = registered assets / required assets
--

Failure mode: The agent invents brand facts or uses an unapproved image.

Control: Limit reasoning to named references and a manifest of authorized assets.

Acceptance evidence: asset-manifest.csv resolves every asset and source identifier.

2.10 Action, motion and environment

Mechanism

- 1. Name subject
- 2. Choose action
- 3. Choose camera
- 4. Lock environment

subject	action	camera_motion	environment
Required state field for name subject	Required state field for choose action	Required state field for choose camera	Required state field for lock environment

Evidence and control

MEASURE usable shot rate = accepted shots / attempts
--

Failure mode: One prompt combines multiple actions and conflicting camera moves.

Control: Use one dominant subject action and one camera move per timed shot.

Acceptance evidence: shot-list.json shows one action-motion pair per shot.

2.11 Continuity and negative constraints

Mechanism

- 1. Lock identity
- 2. Lock palette
- 3. List invariants
- 4. Reject drift

continuity_token	palette_hex	must_keep	must_avoid
Required state field for lock identity	Required state field for lock palette	Required state field for list invariants	Required state field for reject drift

Evidence and control

MEASURE
continuity defects per reviewed shot

Failure mode: Characters, products or brand colors drift between generated clips.

Control: Reuse continuity tokens and compare keyframes against explicit invariants.

Acceptance evidence: continuity-review.json records deviations and acceptance.

2.12 Structured shot-plan output

Mechanism

- 1. Request JSON
- 2. Parse response
- 3. Check timeline
- 4. Approve plan

shot_id	start_s	end_s	prompt
Required state field for request json	Required state field for parse response	Required state field for check timeline	Required state field for approve plan

Evidence and control

MEASURE timeline variance = planned duration - target duration
--

Failure mode: Plausible prose cannot be handed reliably to render tools.

Control: Require strict JSON and fail on gaps, overlaps or unknown fields.

Acceptance evidence: shot-plan.json totals the target duration with no overlap.

2.13 Simple video agent loop

Mechanism

- 1. Receive brief
- 2. Plan shots
- 3. Call tool
- 4. Inspect result

run_id	plan_hash	tool_call	result_uri
Required state field for receive brief	Required state field for plan shots	Required state field for call tool	Required state field for inspect result

Evidence and control

MEASURE first-pass completion rate
--

Failure mode: The agent repeatedly regenerates without a finish condition.

Control: Set one render attempt, one review pass and an explicit repair decision.

Acceptance evidence: run-ledger.json contains the plan, call, result and stop reason.

2.14 Cost, latency and stop rules

Mechanism

- 1. Estimate calls
- 2. Reserve retries
- 3. Set ceiling
- 4. Stop or escalate

estimated_cost	max_attempts	timeout_s	stop_reason
Required state field for estimate calls	Required state field for reserve retries	Required state field for set ceiling	Required state field for stop or escalate

Evidence and control

MEASURE budget variance = actual cost - approved cost

Failure mode: A trial quota is consumed by open-ended retries.

Control: Cap attempts and require a human decision before additional paid generation.

Acceptance evidence: budget-ledger.csv reconciles attempts, latency and cost.

3. Video Tools, Hermes Skills and Custom Brand Production

LO2 - create a technically valid branded video by combining agent reasoning with specialist media tools

TOPIC OPERATING PRINCIPLE

Every agent handoff uses named fields; every consequential transition has a deterministic gate; every release decision resolves to evidence.

3.1 Tool versus skill decision

Mechanism

- 1. Identify capability
- 2. Assess auth
- 3. Choose tool
- 4. Wrap procedure

capability	auth_mode	tool_or_skill	owner
Required state field for identify capability	Required state field for assess auth	Required state field for choose tool	Required state field for wrap procedure

Evidence and control

MEASURE

integration fit = satisfied requirements / requirements

Failure mode: A brittle instruction-only skill tries to manage streaming media or complex authentication.

Control: Use a tool for precise integrated execution and a skill for repeatable instructions plus scripts.

Acceptance evidence: decision-record.md states why each capability is a tool or skill.

3.2 Hermes skill anatomy

Mechanism

- 1. Create directory
- 2. Write frontmatter
- 3. Add procedure
- 4. Test invocation

name	description	version	skill_dir
Required state field for create directory	Required state field for write frontmatter	Required state field for add procedure	Required state field for test invocation

Evidence and control

MEASURE skill test pass = expected artifacts produced

Failure mode: A vague description prevents Hermes from discovering the skill.

Control: Describe the exact trigger and include deterministic verification commands.

Acceptance evidence: SKILL.md and test transcript show correct activation.

3.3 Skill discovery and progressive loading

Mechanism

- 1. List skills
- 2. Search catalog
- 3. View selected skill
- 4. Invoke task

skill_name	category	file_path	task
Required state field for list skills	Required state field for search catalog	Required state field for view selected skill	Required state field for invoke task

Evidence and control

MEASURE context efficiency = loaded relevant files / loaded files

Failure mode: The agent loads a large library or the wrong similarly named skill.

Control: Search first, view the selected skill, then load only referenced resources.

Acceptance evidence: skill-usage.json records the selected skill and resources.

3.4 Remotion composition contract

Mechanism

- 1. Define composition
- 2. Bind props
- 3. Preview frames
- 4. Render MP4

composition_id	fps	duration_frames	input_props
Required state field for define composition	Required state field for bind props	Required state field for preview frames	Required state field for render mp4

Evidence and control

MEASURE duration seconds = duration frames / fps
--

Failure mode: The render duration or aspect ratio drifts from the approved video specification.

Control: Derive frames from duration and validate width, height, fps and codec after render.

Acceptance evidence: remotion-render.json and ffprobe.json agree with the contract.

3.5 Manim explanatory scene

Mechanism

- 1. Define objects
- 2. Stage transforms
- 3. Render scene
- 4. Composite clip

scene_class	resolution	frame_rate	output_file
Required state field for define objects	Required state field for stage transforms	Required state field for render scene	Required state field for composite clip

Evidence and control

MEASURE animation timing variance

Failure mode: A mathematical or process animation is visually attractive but semantically wrong.

Control: Bind labels and values to the approved evidence model and review rendered frames.

Acceptance evidence: manim-scene.py and contact sheet prove the intended transformation.

3.6 Higgsfield shot request

Mechanism

- 1. Select model
- 2. Attach reference
- 3. Write motion prompt
- 4. Review output

model	reference_uri	motion_prompt	generation_id
Required state field for select model	Required state field for attach reference	Required state field for write motion prompt	Required state field for review output

Evidence and control

MEASURE accepted clip yield = approved clips / generated clips
--

Failure mode: A cloud generation call consumes quota before prompt and rights checks pass.

Control: Create a request preview first; call the service only with approved assets and quota.

Acceptance evidence: higgsfield-request.json records prompt, references and review status.

3.7 FFmpeg deterministic assembly

Mechanism

- 1. Normalize media
- 2. Build timeline
- 3. Mix audio
- 4. Encode master

filter_complex	video_codec	audio_codec	pix_fmt
Required state field for normalize media	Required state field for build timeline	Required state field for mix audio	Required state field for encode master

Evidence and control

MEASURE render success = valid masters / render attempts
--

Failure mode: Incompatible dimensions, frame rates or time bases break composition.

Control: Normalize every source before concat and probe the final container.

Acceptance evidence: ffmpegprobe.json confirms H.264, AAC, dimensions, fps and duration.

3.8 Tool permission boundary

Mechanism

- 1. Declare capability
- 2. Set working root
- 3. Request approval
- 4. Record result

allowed_paths	network_scope	side_effect	approval_id
Required state field for declare capability	Required state field for set working root	Required state field for request approval	Required state field for record result

Evidence and control

MEASURE
unauthorized side effects = 0

Failure mode: A broad terminal or browser tool can publish or overwrite the wrong artifact.

Control: Scope paths and accounts; separate preview from consequential execution.

Acceptance evidence: tool-policy.yaml and audit log show bounded operations.

3.9 Asset provenance and checksum

Mechanism

- 1. Register source
- 2. Record license
- 3. Compute hash
- 4. Freeze version

asset_id	source_uri	rights_status	sha256
Required state field for register source	Required state field for record license	Required state field for compute hash	Required state field for freeze version

Evidence and control

MEASURE manifest integrity = verified hashes / manifest rows
--

Failure mode: A reviewed asset changes while retaining the same filename.

Control: Bind approval to cryptographic hashes and immutable versions.

Acceptance evidence: asset-manifest.csv verifies rights and hashes.

3.10 Brand token system

Mechanism

- 1. Extract identity
- 2. Set palette
- 3. Set typography
- 4. Set motion rules

logo_uri	palette_hex	font_stack	motion_style
Required state field for extract identity	Required state field for set palette	Required state field for set typography	Required state field for set motion rules

Evidence and control

MEASURE brand compliance = passed checks / applicable checks
--

Failure mode: The agent improvises colors, tone or animation on every run.

Control: Store editable brand tokens and reference them from the video skill.

Acceptance evidence: brand-profile.yaml and frame samples pass the checklist.

3.11 Tone and style controls

Mechanism

- 1. Define audience
- 2. Set voice
- 3. Set pacing
- 4. Define exclusions

audience	tone	words_per_min	style_avoid
Required state field for define audience	Required state field for set voice	Required state field for set pacing	Required state field for define exclusions

Evidence and control

MEASURE
tone adherence score

Failure mode: A polished output sounds off-brand or imitates a protected creator style.

Control: Use descriptive attributes and exclusions, not living-artist imitation.

Acceptance evidence: review rubric records tone, pacing and originality evidence.

3.12 Custom brand-video skill

Mechanism

- 1. Write skill
- 2. Bundle scripts
- 3. Add templates
- 4. Run acceptance test

skill_name	script_path	template_path	output_contract
Required state field for write skill	Required state field for bundle scripts	Required state field for add templates	Required state field for run acceptance test

Evidence and control

MEASURE repeatability = matching outputs / repeated runs
--

Failure mode: The skill depends on hidden context or absolute project paths.

Control: Use Hermes template variables and explicit inputs, outputs and verification.

Acceptance evidence: custom skill produces the expected video and evidence on a clean run.

3.13 Technical video quality gate

Mechanism

1. Probe container
2. Inspect frames
3. Check captions
4. Gate master

width	height	avg_frame_rate	duration_s
Required state field for probe container	Required state field for inspect frames	Required state field for check captions	Required state field for gate master

Evidence and control

MEASURE

technical checks passed / technical checks

Failure mode: A video plays locally but violates channel or accessibility requirements.

Control: Fail closed on missing streams, wrong bounds or unreadable captions.

Acceptance evidence: technical-qc.json and contact sheet show observed values.

3.14 Repair and versioned evidence

Mechanism

- 1. Open finding
- 2. Assign repair
- 3. Render version
- 4. Re-test scope

finding_id	severity	asset_version	resolution
Required state field for open finding	Required state field for assign repair	Required state field for render version	Required state field for re-test scope

Evidence and control

MEASURE

repair effectiveness = closed findings / reopened findings

Failure mode: A repair overwrites the reviewed master or introduces a new defect.

Control: Write a new immutable version and rerun every affected check.

Acceptance evidence: repair-log.json links finding, change, hash and re-test.

4. Multi-Agent Kanban, YouTube Release and Scheduled Publishing

LO3 - operate a durable role-based production system with review, release and scheduling safeguards

TOPIC OPERATING PRINCIPLE

Every agent handoff uses named fields; every consequential transition has a deterministic gate; every release decision resolves to evidence.

4.1 Agent role contract

Mechanism

- 1. Define goal
- 2. Limit tools
- 3. Name output
- 4. Set finish rule

agent_name	allowed_tools	deliverable	done_when
Required state field for define goal	Required state field for limit tools	Required state field for name output	Required state field for set finish rule

Evidence and control

MEASURE

role completeness = required fields / required fields

Failure mode: A role such as video agent is too broad to evaluate or recover.

Control: Give every agent one owned deliverable and one measurable completion rule.

Acceptance evidence: agent-contracts.yaml validates all four specialist roles.

4.2 Research agent

Mechanism

- 1. Read brief
- 2. Retrieve sources
- 3. Score evidence
- 4. Handoff claims

research_task	source_ids	claim_ids	handoff_status
Required state field for read brief	Required state field for retrieve sources	Required state field for score evidence	Required state field for handoff claims

Evidence and control

MEASURE claim coverage = sourced claims / factual claims
--

Failure mode: Research output contains fabricated or irretrievable citations.

Control: Require retrievable URLs, bounded excerpts and explicit limitations.

Acceptance evidence: research-handoff.json contains approved sources and claim IDs.

4.3 Video production agent

Mechanism

- 1. Read approved claims
- 2. Create shot plan
- 3. Invoke skills
- 4. Return master

claim_ids	shot_plan	skill_names	master_hash
Required state field for read approved claims	Required state field for create shot plan	Required state field for invoke skills	Required state field for return master

Evidence and control

MEASURE production acceptance = passed render checks / checks

Failure mode: The production agent changes approved claims or bypasses brand controls.

Control: Provide immutable research input and require the custom brand-video skill.

Acceptance evidence: video-handoff.json references approved claims and master hash.

4.4 Independent review agent

Mechanism

- 1. Inspect evidence
- 2. Score rubric
- 3. Create findings
- 4. Request decision

master_hash	rubric_version	findings	qa_status
Required state field for inspect evidence	Required state field for score rubric	Required state field for create findings	Required state field for request decision

Evidence and control

MEASURE weighted review score and unresolved high count

Failure mode: The same agent that created the video self-approves its output.

Control: Use an independent context and forbid approval authority in the reviewer role.

Acceptance evidence: review-handoff.json contains timecoded findings and QA status.

4.5 Upload agent

Mechanism

- 1. Verify approval
- 2. Build metadata
- 3. Upload private
- 4. Record video ID

approval_hash	privacy_status	synthetic_media	video_id
Required state field for verify approval	Required state field for build metadata	Required state field for upload private	Required state field for record video id

Evidence and control

MEASURE upload reconciliation = one video ID per idempotency key
--

Failure mode: A video is uploaded publicly or with stale approval.

Control: Default to private and verify the current master hash immediately before upload.

Acceptance evidence: upload-receipt.json records privacy, disclosure and returned video ID.

4.6 Delegation context contract

Mechanism

- 1. Package context
- 2. Dispatch child
- 3. Receive summary
- 4. Verify artifact

goal	context	paths	acceptance
Required state field for package context	Required state field for dispatch child	Required state field for receive summary	Required state field for verify artifact

Evidence and control

MEASURE handoff pass rate = accepted handoffs / handoffs
--

Failure mode: A subagent starts fresh and lacks the files, constraints or error state it needs.

Control: Include exact paths, inputs, constraints and tests in every delegation context.

Acceptance evidence: delegation-log.json records prompt, agent and verified result.

4.7 Parallel work and join

Mechanism

- 1. Find independent tasks
- 2. Dispatch batch
- 3. Track results
- 4. Join after checks

task_ids	dependencies	result_refs	join_status
Required state field for find independent tasks	Required state field for dispatch batch	Required state field for track results	Required state field for join after checks

Evidence and control

MEASURE elapsed reduction with zero skipped dependencies
--

Failure mode: Parallel agents race on shared files or a join proceeds with missing output.

Control: Parallelize only independent work and join on explicit verified artifacts.

Acceptance evidence: join-ledger.json shows all required parents complete.

4.8 Durable Hermes Kanban

Mechanism

- 1. Create tasks
- 2. Assign profiles
- 3. Link dependencies
- 4. Run dispatcher

task_id	assignee	status	parent_ids
Required state field for create tasks	Required state field for assign profiles	Required state field for link dependencies	Required state field for run dispatcher

Evidence and control

MEASURE flow time = done_at - created_at
--

Failure mode: An in-process agent swarm loses state when a session ends.

Control: Use Kanban for cross-agent work that must survive restarts and human pauses.

Acceptance evidence: kanban-export.json shows the full dependency chain and statuses.

4.9 Kanban status and circuit breaker

Mechanism

- 1. Move to ready
- 2. Run worker
- 3. Request review
- 4. Block or complete

status	attempt	failure_class	reviewer
Required state field for move to ready	Required state field for run worker	Required state field for request review	Required state field for block or complete

Evidence and control

MEASURE retry recovery and repeated-failure count

Failure mode: A failing task retries indefinitely or is marked done from a commit alone.

Control: Cap attempts, require contract verification and block repeated failures for intervention.

Acceptance evidence: task history shows checkpoints, review and final evidence.

4.10 Human approval with payload hash

Mechanism

- 1. Freeze package
- 2. Compute hash
- 3. Request approval
- 4. Recheck before upload

approval_id	payload_hash	reviewer	decision
Required state field for freeze package	Required state field for compute hash	Required state field for request approval	Required state field for recheck before upload

Evidence and control

MEASURE approval integrity = approved hash equals current hash
--

Failure mode: A reviewer approves one version and the upload agent sends another.

Control: Bind the named decision to the immutable release package hash.

Acceptance evidence: approval-ledger.json proves reviewer, time, decision and hash.

4.11 YouTube videos.insert contract

Mechanism

- 1. Authorize OAuth
- 2. Set snippet
- 3. Set status
- 4. Upload media

snippet.title	status.privacyStatus	status.containsSyntheticMedia	media_path
Required state field for authorize oauth	Required state field for set snippet	Required state field for set status	Required state field for upload media

Evidence and control

MEASURE

metadata completeness and processing success

Failure mode: The wrong account, privacy state or disclosure reaches YouTube.

Control: Confirm channel identity, default private and include applicable synthetic-media disclosure.

Acceptance evidence: request-preview.json and private upload receipt match the approved package.

4.12 Publishing idempotency

Mechanism

- 1. Create publish key
- 2. Check prior result
- 3. Attempt upload
- 4. Commit external ID

idempotency_key	channel_id	video_id	published_at
Required state field for create publish key	Required state field for check prior result	Required state field for attempt upload	Required state field for commit external id

Evidence and control

MEASURE duplicate publish rate = 0
--

Failure mode: A timeout hides a successful upload and retry creates a duplicate.

Control: Reconcile the prior key and video ID before any retry.

Acceptance evidence: publication-ledger.csv contains one committed ID per key.

4.13 Hermes cron job contract

Mechanism

- 1. Write self-contained prompt
- 2. Attach skills
- 3. Create schedule
- 4. Inspect next run

job_name	schedule	skills	delivery
Required state field for write self-contained prompt	Required state field for attach skills	Required state field for create schedule	Required state field for inspect next run

Evidence and control

MEASURE
schedule validity and successful dry run

Failure mode: A cron session starts fresh and lacks the approved brief, paths or account context.

Control: Put all required paths, gates and stop rules in the job prompt; attach skills explicitly.

Acceptance evidence: cron-preview.json records schedule, next run and paused state.

4.14 Scheduled release operations

Mechanism

- 1. Keep job paused
- 2. Trigger dry run
- 3. Review output
- 4. Enable cadence

paused	last_run	next_run	owner
Required state field for keep job paused	Required state field for trigger dry run	Required state field for review output	Required state field for enable cadence

Evidence and control

MEASURE on-time success rate with zero unauthorized posts

Failure mode: A recurring job publishes an unreviewed or outdated video automatically.

Control: Schedule preparation and private upload only; require approval before public visibility.

Acceptance evidence: operations-ledger.csv records each run, outcome and owner decision.

5. Hands-on Lab Guide

Each lab is a self-contained Hermes project with copy-ready prompts in Markdown and PDF, synthetic inputs, scripts or configuration, an evidence checklist and a deterministic verifier.

5.1 Lab 01: Set Up Hermes Desktop and Connect MiniMax M3

OBJECTIVE

Install Hermes Desktop from the official source, configure the MiniMax provider for MiniMax-M3 and prove a tool-capable model handshake without exposing a credential.

Folder contents

- labs/lab-01-setup-hermes-and-connect-minimax-m3/README.md — detailed procedure
- AI-PROMPTS.md and AI-PROMPTS.pdf — identical copy-ready prompts
- data/ — synthetic input, configuration or orchestration records
- starter/ and skills/ — reusable commands, scripts and Hermes skill examples
- verify.py plus evidence/checklist.md — deterministic and human evidence gates

Detailed procedure

1. Open the lab folder as the current project in Hermes Desktop and confirm the active session model.
2. Read README.md and inspect every supplied input, script, configuration and expected output before asking Hermes to act.
3. Open AI-PROMPTS.md or AI-PROMPTS.pdf and replace only the named placeholders with supplied synthetic values.
4. Ask Hermes for a plan that names files, tools, skills, side effects, approval points, acceptance tests and stop conditions.
5. Keep network, paid generation, YouTube upload and scheduling in preview or paused mode unless the trainer authorizes the exact action.
6. Run the task and follow run_id, source IDs, asset hashes and status fields through each stage.
7. Run python3 verify.py and compare observed evidence with the lab acceptance criterion.
8. Trigger one controlled failure, confirm the gate blocks progress, restore the approved input and rerun the verifier.
9. Complete evidence/checklist.md and retain only the screenshots, logs and files requested for assessment.
10. Sanitize learner evidence: remove credentials, account IDs, tokens and unrelated personal or execution data.

Lab-specific execution checks

Sequence	Hermes stage	What to inspect
1	Download official installer	Inputs, allowed tools, output evidence, explicit status and stop condition
2	Complete Hermes setup	Inputs, allowed tools, output evidence, explicit status and stop condition
3	Configure MiniMax-M3	Inputs, allowed tools, output evidence, explicit status and stop condition
4	Run hermes doctor	Inputs, allowed tools, output evidence, explicit status and stop condition
5	Verify model response	Inputs, allowed tools, output evidence, explicit status and stop condition

6	Redact evidence	Inputs, allowed tools, output evidence, explicit status and stop condition
---	-----------------	--

Prompt contract

Act as my Hermes setup verifier. Inspect only the redacted configuration and diagnostic output I provide. Return JSON with provider, model, doctor_status, missing_requirements, security_findings and next_action. Require model MiniMax-M3. Never ask me to paste an API key.

Acceptance check

PASS ONLY WHEN Hermes launches; the configured provider is minimax; the model is MiniMax-M3; hermes doctor has no blocking failure; published evidence contains placeholders only.
--

Troubleshooting

Observed issue	Likely cause	Remedial action
Node receives undefined	Upstream key name changed	Compare execution JSON to the node contract; restore the required key.
Workflow loops or duplicates	No stop reason or prior-key check	Restore max attempts and idempotency lookup before the side effect.
Output is plausible but unsupported	Prompt not bounded to approved evidence	Block the run and attach a valid source_id/claim_id before regeneration.
Approval does not release	Current payload hash differs	Return to review; approve the current immutable payload only.

5.2 Lab 02: Prompt Hermes to Create a Simple Video

OBJECTIVE

Use a copy-ready Hermes prompt to convert a supplied 15-second brief into a validated shot plan and deterministic preview video.

Folder contents

- labs/lab-02-prompt-hermes-to-create-a-simple-video/README.md — detailed procedure
- AI-PROMPTS.md and AI-PROMPTS.pdf — identical copy-ready prompts
- data/ — synthetic input, configuration or orchestration records
- starter/ and skills/ — reusable commands, scripts and Hermes skill examples
- verify.py plus evidence/checklist.md — deterministic and human evidence gates

Detailed procedure

1. Open the lab folder as the current project in Hermes Desktop and confirm the active session model.
2. Read README.md and inspect every supplied input, script, configuration and expected output before asking Hermes to act.
3. Open AI-PROMPTS.md or AI-PROMPTS.pdf and replace only the named placeholders with supplied synthetic values.
4. Ask Hermes for a plan that names files, tools, skills, side effects, approval points, acceptance tests and stop conditions.
5. Keep network, paid generation, YouTube upload and scheduling in preview or paused mode unless the trainer authorizes the exact action.
6. Run the task and follow run_id, source IDs, asset hashes and status fields through each stage.
7. Run python3 verify.py and compare observed evidence with the lab acceptance criterion.
8. Trigger one controlled failure, confirm the gate blocks progress, restore the approved input and rerun the verifier.
9. Complete evidence/checklist.md and retain only the screenshots, logs and files requested for assessment.
10. Sanitize learner evidence: remove credentials, account IDs, tokens and unrelated personal or execution data.

Lab-specific execution checks

Sequence	Hermes stage	What to inspect
1	Open project folder	Inputs, allowed tools, output evidence, explicit status and stop condition
2	Submit bounded prompt	Inputs, allowed tools, output evidence, explicit status and stop condition
3	Validate shot plan	Inputs, allowed tools, output evidence, explicit status and stop condition
4	Run preview renderer	Inputs, allowed tools, output evidence, explicit status and stop condition
5	Probe MP4	Inputs, allowed tools, output evidence, explicit status and stop condition
6	Record evidence	Inputs, allowed tools, output evidence, explicit status and stop condition

Prompt contract

Create a 15-second 1080x1920 MP4 for Harbour Bean using only the supplied brief and assets. First return shot-plan.json with three non-overlapping five-second shots. Then run starter/render_preview.py. Do not use network tools. Stop when verify.py reports PASS and return the output path plus observed ffprobe values.

Acceptance check

PASS ONLY WHEN

The shot plan is valid JSON, totals 15 seconds, uses only supplied assets, and the generated MP4 passes dimensions, codec and duration checks.

Troubleshooting

Observed issue	Likely cause	Remedial action
Node receives undefined	Upstream key name changed	Compare execution JSON to the node contract; restore the required key.
Workflow loops or duplicates	No stop reason or prior-key check	Restore max attempts and idempotency lookup before the side effect.
Output is plausible but unsupported	Prompt not bounded to approved evidence	Block the run and attach a valid source_id/claim_id before regeneration.
Approval does not release	Current payload hash differs	Return to review; approve the current immutable payload only.

5.3 Lab 03: Engineer Video Prompts with FRAME-CUT

OBJECTIVE

Transform a vague creative request into portable shot prompts using the FRAME-CUT framework and evaluate them with a deterministic rubric.

Folder contents

- labs/lab-03-engineer-video-prompts-with-frame-cut/README.md — detailed procedure
- AI-PROMPTS.md and AI-PROMPTS.pdf — identical copy-ready prompts
- data/ — synthetic input, configuration or orchestration records
- starter/ and skills/ — reusable commands, scripts and Hermes skill examples
- verify.py plus evidence/checklist.md — deterministic and human evidence gates

Detailed procedure

1. Open the lab folder as the current project in Hermes Desktop and confirm the active session model.
2. Read README.md and inspect every supplied input, script, configuration and expected output before asking Hermes to act.
3. Open AI-PROMPTS.md or AI-PROMPTS.pdf and replace only the named placeholders with supplied synthetic values.
4. Ask Hermes for a plan that names files, tools, skills, side effects, approval points, acceptance tests and stop conditions.
5. Keep network, paid generation, YouTube upload and scheduling in preview or paused mode unless the trainer authorizes the exact action.
6. Run the task and follow run_id, source IDs, asset hashes and status fields through each stage.
7. Run python3 verify.py and compare observed evidence with the lab acceptance criterion.
8. Trigger one controlled failure, confirm the gate blocks progress, restore the approved input and rerun the verifier.
9. Complete evidence/checklist.md and retain only the screenshots, logs and files requested for assessment.
10. Sanitize learner evidence: remove credentials, account IDs, tokens and unrelated personal or execution data.

Lab-specific execution checks

Sequence	Hermes stage	What to inspect
1	Diagnose vague prompt	Inputs, allowed tools, output evidence, explicit status and stop condition
2	Complete FRAME fields	Inputs, allowed tools, output evidence, explicit status and stop condition
3	Complete CUT fields	Inputs, allowed tools, output evidence, explicit status and stop condition
4	Generate shot JSON	Inputs, allowed tools, output evidence, explicit status and stop condition
5	Score rubric	Inputs, allowed tools, output evidence, explicit status and stop condition
6	Repair one defect	Inputs, allowed tools, output evidence, explicit status and stop condition

Prompt contract

Apply FRAME-CUT. FRAME = Format and finish; Role and references; Assets and action; Motion; Environment and exposure. CUT = Continuity; Unwanted changes; Technical output. Return one JSON object per shot with every field, duration, seed_policy and acceptance_check. Do not imitate a living artist or invent brand facts.

Acceptance check

PASS ONLY WHEN

Every shot has all FRAME-CUT fields, one dominant action, one camera move, explicit continuity and negative constraints, and a measurable output check.

Troubleshooting

Observed issue	Likely cause	Remedial action
Node receives undefined	Upstream key name changed	Compare execution JSON to the node contract; restore the required key.
Workflow loops or duplicates	No stop reason or prior-key check	Restore max attempts and idempotency lookup before the side effect.
Output is plausible but unsupported	Prompt not bounded to approved evidence	Block the run and attach a valid source_id/claim_id before regeneration.
Approval does not release	Current payload hash differs	Return to review; approve the current immutable payload only.

5.4 Lab 04: Install Video Tools and Hermes Skills

OBJECTIVE

Create a governed tool registry and install learner-safe Hermes skills for Remotion, Manim, Higgsfield request preparation and FFmpeg verification.

Folder contents

- labs/lab-04-install-video-tools-and-skills/README.md — detailed procedure
- AI-PROMPTS.md and AI-PROMPTS.pdf — identical copy-ready prompts
- data/ — synthetic input, configuration or orchestration records
- starter/ and skills/ — reusable commands, scripts and Hermes skill examples
- verify.py plus evidence/checklist.md — deterministic and human evidence gates

Detailed procedure

1. Open the lab folder as the current project in Hermes Desktop and confirm the active session model.
2. Read README.md and inspect every supplied input, script, configuration and expected output before asking Hermes to act.
3. Open AI-PROMPTS.md or AI-PROMPTS.pdf and replace only the named placeholders with supplied synthetic values.
4. Ask Hermes for a plan that names files, tools, skills, side effects, approval points, acceptance tests and stop conditions.
5. Keep network, paid generation, YouTube upload and scheduling in preview or paused mode unless the trainer authorizes the exact action.
6. Run the task and follow run_id, source IDs, asset hashes and status fields through each stage.
7. Run python3 verify.py and compare observed evidence with the lab acceptance criterion.
8. Trigger one controlled failure, confirm the gate blocks progress, restore the approved input and rerun the verifier.
9. Complete evidence/checklist.md and retain only the screenshots, logs and files requested for assessment.
10. Sanitize learner evidence: remove credentials, account IDs, tokens and unrelated personal or execution data.

Lab-specific execution checks

Sequence	Hermes stage	What to inspect
1	Inventory local tools	Inputs, allowed tools, output evidence, explicit status and stop condition
2	Search Hermes skills	Inputs, allowed tools, output evidence, explicit status and stop condition
3	Install or create skills	Inputs, allowed tools, output evidence, explicit status and stop condition
4	Run smoke tests	Inputs, allowed tools, output evidence, explicit status and stop condition
5	Record permissions	Inputs, allowed tools, output evidence, explicit status and stop condition
6	Choose fallback	Inputs, allowed tools, output evidence, explicit status and stop condition

Prompt contract

Inspect the supplied tool registry. Recommend Remotion for code-driven branded motion graphics, Manim for explanatory diagrams, Higgsfield for approved generative shots, and FFmpeg for deterministic assembly and probing. Use skill_view before invocation. Return a routing table and do not install or call a paid service without approval.

Acceptance check

PASS ONLY WHEN

The registry records capability, installation state, auth mode, side effects and fallback; every supplied SKILL.md passes frontmatter and path checks.

Troubleshooting

Observed issue	Likely cause	Remedial action
Node receives undefined	Upstream key name changed	Compare execution JSON to the node contract; restore the required key.
Workflow loops or duplicates	No stop reason or prior-key check	Restore max attempts and idempotency lookup before the side effect.
Output is plausible but unsupported	Prompt not bounded to approved evidence	Block the run and attach a valid source_id/claim_id before regeneration.
Approval does not release	Current payload hash differs	Return to review; approve the current immutable payload only.

5.5 Lab 05: Create a Custom Branded Video Skill

OBJECTIVE

Turn brand, tone and style rules into a reusable Hermes skill, render a custom video and verify the output against brand and technical evidence.

Folder contents

- labs/lab-05-create-custom-branded-video-skill/README.md — detailed procedure
- AI-PROMPTS.md and AI-PROMPTS.pdf — identical copy-ready prompts
- data/ — synthetic input, configuration or orchestration records
- starter/ and skills/ — reusable commands, scripts and Hermes skill examples
- verify.py plus evidence/checklist.md — deterministic and human evidence gates

Detailed procedure

1. Open the lab folder as the current project in Hermes Desktop and confirm the active session model.
2. Read README.md and inspect every supplied input, script, configuration and expected output before asking Hermes to act.
3. Open AI-PROMPTS.md or AI-PROMPTS.pdf and replace only the named placeholders with supplied synthetic values.
4. Ask Hermes for a plan that names files, tools, skills, side effects, approval points, acceptance tests and stop conditions.
5. Keep network, paid generation, YouTube upload and scheduling in preview or paused mode unless the trainer authorizes the exact action.
6. Run the task and follow run_id, source IDs, asset hashes and status fields through each stage.
7. Run python3 verify.py and compare observed evidence with the lab acceptance criterion.
8. Trigger one controlled failure, confirm the gate blocks progress, restore the approved input and rerun the verifier.
9. Complete evidence/checklist.md and retain only the screenshots, logs and files requested for assessment.
10. Sanitize learner evidence: remove credentials, account IDs, tokens and unrelated personal or execution data.

Lab-specific execution checks

Sequence	Hermes stage	What to inspect
1	Approve brand profile	Inputs, allowed tools, output evidence, explicit status and stop condition
2	Create SKILL.md	Inputs, allowed tools, output evidence, explicit status and stop condition
3	Bind templates	Inputs, allowed tools, output evidence, explicit status and stop condition
4	Render video	Inputs, allowed tools, output evidence, explicit status and stop condition
5	Review frames	Inputs, allowed tools, output evidence, explicit status and stop condition
6	Version evidence	Inputs, allowed tools, output evidence, explicit status and stop condition

Prompt contract

Use the harbour-bean-video skill. Read data/brand-profile.yaml, preserve the exact palette, typography, tone and safe margins, and render a 15-second vertical video from supplied assets. Return the output path, SHA-256, ffmpeg summary and brand-check result. Do not alter the logo or invent slogans.

Acceptance check

PASS ONLY WHEN

The custom skill is discoverable, uses relative or Hermes template paths, creates an MP4, passes the technical probe and meets every required brand token.

Troubleshooting

Observed issue	Likely cause	Remedial action
Node receives undefined	Upstream key name changed	Compare execution JSON to the node contract; restore the required key.
Workflow loops or duplicates	No stop reason or prior-key check	Restore max attempts and idempotency lookup before the side effect.
Output is plausible but unsupported	Prompt not bounded to approved evidence	Block the run and attach a valid source_id/claim_id before regeneration.
Approval does not release	Current payload hash differs	Return to review; approve the current immutable payload only.

5.6 Lab 06: Build the Multi-Agent Video Workflow

OBJECTIVE

Define and simulate four isolated Hermes roles for research, video creation, independent review and approved YouTube upload.

Folder contents

- labs/lab-06-build-multi-agent-video-workflow/README.md — detailed procedure
- AI-PROMPTS.md and AI-PROMPTS.pdf — identical copy-ready prompts
- data/ — synthetic input, configuration or orchestration records
- starter/ and skills/ — reusable commands, scripts and Hermes skill examples
- verify.py plus evidence/checklist.md — deterministic and human evidence gates

Detailed procedure

1. Open the lab folder as the current project in Hermes Desktop and confirm the active session model.
2. Read README.md and inspect every supplied input, script, configuration and expected output before asking Hermes to act.
3. Open AI-PROMPTS.md or AI-PROMPTS.pdf and replace only the named placeholders with supplied synthetic values.
4. Ask Hermes for a plan that names files, tools, skills, side effects, approval points, acceptance tests and stop conditions.
5. Keep network, paid generation, YouTube upload and scheduling in preview or paused mode unless the trainer authorizes the exact action.
6. Run the task and follow run_id, source IDs, asset hashes and status fields through each stage.
7. Run python3 verify.py and compare observed evidence with the lab acceptance criterion.
8. Trigger one controlled failure, confirm the gate blocks progress, restore the approved input and rerun the verifier.
9. Complete evidence/checklist.md and retain only the screenshots, logs and files requested for assessment.
10. Sanitize learner evidence: remove credentials, account IDs, tokens and unrelated personal or execution data.

Lab-specific execution checks

Sequence	Hermes stage	What to inspect
1	Define role contracts	Inputs, allowed tools, output evidence, explicit status and stop condition
2	Package context	Inputs, allowed tools, output evidence, explicit status and stop condition
3	Delegate research	Inputs, allowed tools, output evidence, explicit status and stop condition
4	Delegate production	Inputs, allowed tools, output evidence, explicit status and stop condition
5	Request review	Inputs, allowed tools, output evidence, explicit status and stop condition
6	Gate uploader	Inputs, allowed tools, output evidence, explicit status and stop condition

Prompt contract

Create four specialist roles: researcher, video-producer, independent-reviewer and youtube-uploader. Each role receives exact paths, allowed tools, output schema and done_when. Research and visual preparation may run in parallel; upload depends on verified production, independent review and matching human approval_hash. Return a dependency graph and delegation prompts.

Acceptance check

PASS ONLY WHEN

All roles have bounded tools and outputs; the reviewer is independent; upload is blocked until all parent evidence and the current approval hash pass.

Troubleshooting

Observed issue	Likely cause	Remedial action
Node receives undefined	Upstream key name changed	Compare execution JSON to the node contract; restore the required key.
Workflow loops or duplicates	No stop reason or prior-key check	Restore max attempts and idempotency lookup before the side effect.
Output is plausible but unsupported	Prompt not bounded to approved evidence	Block the run and attach a valid source_id/claim_id before regeneration.
Approval does not release	Current payload hash differs	Return to review; approve the current immutable payload only.

5.7 Lab 07: Orchestrate Kanban Review and YouTube Upload

OBJECTIVE

Create a durable Hermes Kanban dependency chain, review the approved video, and prepare or execute a private YouTube upload with explicit human authorization.

Folder contents

- labs/lab-07-orchestrate-kanban-review-and-youtube-upload/README.md — detailed procedure
- AI-PROMPTS.md and AI-PROMPTS.pdf — identical copy-ready prompts
- data/ — synthetic input, configuration or orchestration records
- starter/ and skills/ — reusable commands, scripts and Hermes skill examples
- verify.py plus evidence/checklist.md — deterministic and human evidence gates

Detailed procedure

1. Open the lab folder as the current project in Hermes Desktop and confirm the active session model.
2. Read README.md and inspect every supplied input, script, configuration and expected output before asking Hermes to act.
3. Open AI-PROMPTS.md or AI-PROMPTS.pdf and replace only the named placeholders with supplied synthetic values.
4. Ask Hermes for a plan that names files, tools, skills, side effects, approval points, acceptance tests and stop conditions.
5. Keep network, paid generation, YouTube upload and scheduling in preview or paused mode unless the trainer authorizes the exact action.
6. Run the task and follow run_id, source IDs, asset hashes and status fields through each stage.
7. Run python3 verify.py and compare observed evidence with the lab acceptance criterion.
8. Trigger one controlled failure, confirm the gate blocks progress, restore the approved input and rerun the verifier.
9. Complete evidence/checklist.md and retain only the screenshots, logs and files requested for assessment.
10. Sanitize learner evidence: remove credentials, account IDs, tokens and unrelated personal or execution data.

Lab-specific execution checks

Sequence	Hermes stage	What to inspect
1	Create board tasks	Inputs, allowed tools, output evidence, explicit status and stop condition
2	Assign profiles	Inputs, allowed tools, output evidence, explicit status and stop condition
3	Link dependencies	Inputs, allowed tools, output evidence, explicit status and stop condition
4	Request review	Inputs, allowed tools, output evidence, explicit status and stop condition
5	Approve exact hash	Inputs, allowed tools, output evidence, explicit status and stop condition
6	Upload private	Inputs, allowed tools, output evidence, explicit status and stop condition

Prompt contract

Create Kanban tasks for research, video, review and upload. Link upload to completed review and a human approval task. Keep the upload task blocked unless approval_hash equals master_hash. Build a YouTube videos.insert request with privacyStatus private and containsSyntheticMedia when applicable. Do not publish publicly.

Acceptance check

PASS ONLY WHEN

The Kanban graph has no missing dependency; review is required; the request points to the approved master; privacy is private; no credential is present in artifacts.

Troubleshooting

Observed issue	Likely cause	Remedial action
Node receives undefined	Upstream key name changed	Compare execution JSON to the node contract; restore the required key.
Workflow loops or duplicates	No stop reason or prior-key check	Restore max attempts and idempotency lookup before the side effect.
Output is plausible but unsupported	Prompt not bounded to approved evidence	Block the run and attach a valid source_id/claim_id before regeneration.
Approval does not release	Current payload hash differs	Return to review; approve the current immutable payload only.

5.8 Lab 08: Schedule Controlled Video Publishing with Hermes Cron

OBJECTIVE

Create a paused Hermes cron job that prepares a release from a self-contained prompt, dry-run it, and enable a cadence only after evidence review.

Folder contents

- labs/lab-08-schedule-controlled-video-publishing/README.md — detailed procedure
- AI-PROMPTS.md and AI-PROMPTS.pdf — identical copy-ready prompts
- data/ — synthetic input, configuration or orchestration records
- starter/ and skills/ — reusable commands, scripts and Hermes skill examples
- verify.py plus evidence/checklist.md — deterministic and human evidence gates

Detailed procedure

1. Open the lab folder as the current project in Hermes Desktop and confirm the active session model.
2. Read README.md and inspect every supplied input, script, configuration and expected output before asking Hermes to act.
3. Open AI-PROMPTS.md or AI-PROMPTS.pdf and replace only the named placeholders with supplied synthetic values.
4. Ask Hermes for a plan that names files, tools, skills, side effects, approval points, acceptance tests and stop conditions.
5. Keep network, paid generation, YouTube upload and scheduling in preview or paused mode unless the trainer authorizes the exact action.
6. Run the task and follow run_id, source IDs, asset hashes and status fields through each stage.
7. Run python3 verify.py and compare observed evidence with the lab acceptance criterion.
8. Trigger one controlled failure, confirm the gate blocks progress, restore the approved input and rerun the verifier.
9. Complete evidence/checklist.md and retain only the screenshots, logs and files requested for assessment.
10. Sanitize learner evidence: remove credentials, account IDs, tokens and unrelated personal or execution data.

Lab-specific execution checks

Sequence	Hermes stage	What to inspect
1	Write self-contained job	Inputs, allowed tools, output evidence, explicit status and stop condition
2	Attach video skill	Inputs, allowed tools, output evidence, explicit status and stop condition
3	Create paused cron	Inputs, allowed tools, output evidence, explicit status and stop condition
4	Trigger dry run	Inputs, allowed tools, output evidence, explicit status and stop condition
5	Inspect next run	Inputs, allowed tools, output evidence, explicit status and stop condition
6	Approve enablement	Inputs, allowed tools, output evidence, explicit status and stop condition

Prompt contract

Create a Hermes cron job named Weekly approved video release with schedule 0 9 * * 2 in Asia/Singapore. Attach the harbour-bean-video skill. The job must select only APPROVED packages, verify hash and idempotency, upload private, and write a receipt. Create it paused, trigger one dry run, and report next_run_at. Never edit jobs.json directly.

Acceptance check

PASS ONLY WHEN

The schedule and timezone are explicit, the prompt is self-contained, the custom skill is attached, the job begins paused, and the dry run cannot publish an unapproved or duplicate video.

Troubleshooting

Observed issue	Likely cause	Remedial action
Node receives undefined	Upstream key name changed	Compare execution JSON to the node contract; restore the required key.
Workflow loops or duplicates	No stop reason or prior-key check	Restore max attempts and idempotency lookup before the side effect.
Output is plausible but unsupported	Prompt not bounded to approved evidence	Block the run and attach a valid source_id/claim_id before regeneration.
Approval does not release	Current payload hash differs	Return to review; approve the current immutable payload only.

6. Hermes and YouTube Release Contract

The learner workflow uses Hermes role contracts and a durable Kanban dependency chain. The upload agent prepares a YouTube videos.insert request only after independent review and a named human approval are bound to the current master hash. OAuth credentials remain in the authorized account connection and never appear in prompts or lab artifacts.

DEFAULT RELEASE POSTURE

Keep YouTube privacy private, declare synthetic media when applicable, require the current approval_id and payload_hash, and reconcile the returned platform identifier before committing the run.

7. Assessment Preparation

The Written Assessment contains five open-ended questions aligned to K1–K5. The Practical Performance contains three tasks covering A1–A6. Use the lab evidence model: observed value, approved threshold, decision and named owner.

8. References

- **Official course page:** <https://www.tertiarycourses.com.sg/wsqa-gentic-ai-for-video-creation.html> — Course identity, duration, outcomes and TSC
- **Hermes Desktop:** <https://hermes-agent.nousresearch.com/desktop> — Official desktop installer and supported operating systems
- **Hermes Agent installation:** <https://hermes-agent.nousresearch.com/docs/getting-started/installation> — Desktop and CLI installation plus diagnostic commands
- **Hermes model configuration:** <https://hermes-agent.nousresearch.com/docs/user-guide/configuring-models> — Provider, model and auxiliary-task configuration
- **MiniMax M3:** <https://www.minimax.io/models/text/m3> — Current MiniMax-M3 model identifier and agentic capabilities
- **Hermes skills:** <https://hermes-agent.nousresearch.com/docs/guides/work-with-skills> — Skill discovery, invocation and custom SKILL.md structure
- **Hermes delegation:** <https://hermes-agent.nousresearch.com/docs/guides/delegation-patterns> — Isolated subagents, explicit context and parallel work
- **Hermes Kanban:** <https://hermes-agent.nousresearch.com/docs/user-guide/features/kanban> — Durable multi-agent task dependencies and review states
- **Hermes scheduled tasks:** <https://hermes-agent.nousresearch.com/docs/user-guide/features/cron> — Cron jobs, attached skills and fresh-session behavior
- **Remotion documentation:** <https://www.remotion.dev/docs> — Code-driven video compositions and rendering
- **Manim documentation:** <https://docs.manim.community/> — Programmatic explanatory animation
- **Higgsfield:** <https://higgsfield.ai/> — Generative video shot production; verify current account and quota terms
- **YouTube videos.insert:** <https://developers.google.com/youtube/v3/docs/videos/insert> — OAuth upload, snippet and status fields
- **AI Video Prompting:** [reference/AI Video Prompting.pdf](#) — Shot grammar, camera movement and continuity patterns
- **Hermes Agent The Complete Developers:** [reference/Hermes Agent The Complete Developers.pdf](#) — Supplied reference for agent architecture, tools, memory and orchestration

Course repository: <https://github.com/tertiarycourses/TGS-2023036088-Agent-ic-AI-for-Video-Creation>

Learning platform: <https://lms-tms.tertiaryinfotech.com/>